

comparison), and insert it in front of the 3, so the list now reads 2, 3, 4, 5, 1, 6. Find the location for the 4 (compare it to the 2 and then the 3), and insert it, leaving 2, 3, 4, 5, 1, 6. Find the location for the 5 (compare it to the 3 and then the 4), and insert it, leaving 2, 3, 4, 5, 1, 6. Find the location for the 1 (compare it to the 3 and then the 2 and then the 2 again), and insert it, leaving 1, 2, 3, 4, 5, 6. Find the location for the 6 (compare it to the 3 and then the 4 and then the 5), and insert it, giving the final answer 1, 2, 3, 4, 5, 6.

51. procedure *binary insertion sort*($a_1, a_2, \dots, a_n$;
real numbers with $n \geq 2$)
for $j := 2$ **to** n
 {binary search for insertion location i }
 $left := 1$
 $right := j - 1$
 while $left < right$
 $middle := \lfloor (left + right)/2 \rfloor$
 if $a_j > a_{middle}$ **then** $left := middle + 1$
 else $right := middle$
 if $a_j < a_{left}$ **then** $i := left$ **else** $i := left + 1$
 {insert a_j in location i by moving a_i through a_{j-1}
 toward back of list}
 $m := a_j$
 for $k := 0$ **to** $j - i - 1$
 $a_{j-k} := a_{j-k-1}$
 $a_i := m$
 { $a_1, a_2, \dots, a_n$ are sorted}

53. The variation from Exercise 52 **55.** $m = 3, n = 8; s = 0, j = 1$, no match; $s = 1, j = 1, j = 2, j = 3$, no match; $s = 2, j = 1$, no match; $s = 3, j = 1, j = 2$, no match; $s = 4, j = 1$, no match; $s = 5, j = 1, j = 2, j = 3, j = 4$, “5 is a valid shift” **57. a)** Two quarters, one penny **b)** Two quarters, one dime, one nickel, four pennies **c)** Three quarters, one penny **d)** Two quarters, one dime **59.** Cashier’s algorithm uses fewest coins in parts (a), (c), and (d). **a)** Two quarters, one penny **b)** Two quarters, one dime, nine pennies **c)** Three quarters, one penny **d)** Two quarters, one dime
61. The 9:00–9:45 talk, the 9:50–10:15 talk, the 10:15–10:45 talk, the 11:00–11:15 talk **63. a)** Order the talks by starting time. Number the lecture halls 1, 2, 3, and so on. For each talk, assign it to lowest numbered lecture hall that is currently available. **b)** If this algorithm uses n lecture halls, then at the point the n th hall was first assigned, it *had* to be used (otherwise a lower-numbered hall would have been assigned), which means that n talks were going on simultaneously (this talk just assigned and the $n - 1$ talks currently in halls 1 through $n - 1$). **65.** Here we assume that the men are the suitors and the women the suitees.

procedure *stable*($M_1, M_2, \dots, M_s, W_1, W_2, \dots, W_s$;
preference lists)
for $i := 1$ **to** s
 mark man i as rejected
for $i := 1$ **to** s
 set man i ’s rejection list to be empty
for $j := 1$ **to** s
 set woman j ’s proposal list to be empty

while rejected men remain

for $i := 1$ **to** s
 if man i is marked rejected **then** add i to the
 proposal list for the woman j who ranks highest
 on his preference list but does not appear on his
 rejection list, and mark i as not rejected

for $j := 1$ **to** s
 if woman j ’s proposal list is nonempty **then**
 remove from j ’s proposal list all men i
 except the man i_0 who ranks highest on her
 preference list, and for each such man i mark
 him as rejected and add j to his rejection list

for $j := 1$ **to** s
 match j with the one man on j ’s proposal list
 {This matching is stable.}

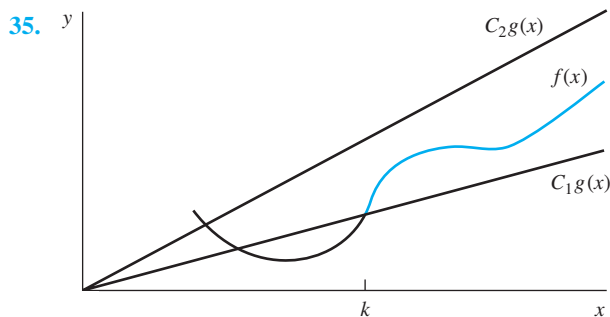
67. If the assignment is not stable, then there is a man m and a woman w such that m prefers w to the woman w' with whom he is matched, and w prefers m to the man with whom she is matched. But m must have proposed to w before he proposed to w' , because he prefers the former. Because m did not end up matched with w , she must have rejected him. Women reject a suitor only when they get a better proposal, and they eventually get matched with a pending suitor, so the woman with whom w is matched must be better in her eyes than m , contradicting our original assumption. Therefore, the marriage is stable. **69. a)** (adapted from Wikipedia) Let m be the majority element. Let n be a number defined at any step of the algorithm to be either the counter, if the majority candidate is m , or the negative of the counter otherwise. Then at each step in which the algorithm encounters a value equal to m , the value of n will increase by 1, and at each step at which it encounters a different value, the value of n may either increase or decrease by one. If m truly is the majority element, there will be more increases than decreases, and n will be positive at the end of the algorithm. But this can be true only when the final stored element is m , the majority element. **b)** A counterexample is ABABC. **71.** Run the two programs on their inputs concurrently and report which one halts.

Section 3.2

1. The choices of C and k are not unique. **a)** $C = 1, k = 10$ **b)** $C = 4, k = 7$ **c)** No **d)** $C = 5, k = 1$ **e)** $C = 1, k = 0$ **f)** $C = 1, k = 2$ **3.** $x^4 + 9x^3 + 4x + 7 \leq 4x^4$ for all $x > 9$; witnesses $C = 4, k = 9$ **5.** $(x^2 + 1)/(x + 1) = x - 1 + 2/(x + 1) < x$ for all $x > 1$; witnesses $C = 1, k = 1$ **7.** The choices of C and k are not unique. **a)** $n = 3, C = 3, k = 1$ **b)** $n = 3, C = 4, k = 1$ **c)** $n = 1, C = 2, k = 1$ **d)** $n = 0, C = 2, k = 1$ **9.** $x^2 + 4x + 17 \leq 3x^3$ for all $x > 17$, so $x^2 + 4x + 17$ is $O(x^3)$, with witnesses $C = 3, k = 17$. However, if x^3 were $O(x^2 + 4x + 17)$, then $x^3 \leq C(x^2 + 4x + 17) \leq 3Cx^2$ for some C , for all sufficiently large x , which implies that $x \leq 3C$ for all sufficiently large x , which is impossible. Hence, x^3 is not $O(x^2 + 4x + 17)$. **11.** $3x^4 + 1 \leq 4x^4 = 8(x^4/2)$ for all $x > 1$, so $3x^4 + 1$ is $O(x^4/2)$, with witnesses $C = 8, k = 1$. Also $x^4/2 \leq 3x^4 + 1$ for all $x > 0$, so $x^4/2$ is

S-22 Answers to Odd-Numbered Exercises

$O(3x^4 + 1)$, with witnesses $C = 1, k = 0$. **13.** Because $2^n \leq 3^n$ for all $n > 0$, it follows that 2^n is $O(3^n)$, with witnesses $C = 1, k = 0$. However, if 3^n were $O(2^n)$, then for some $C, 3^n \leq C \cdot 2^n$ for all sufficiently large n . This says that $C \geq (3/2)^n$ for all sufficiently large n , which is impossible. Hence, 3^n is not $O(2^n)$. **15.** All functions for which there exist real numbers k and C with $|f(x)| \leq C$ for $x > k$. These are the functions $f(x)$ that are bounded for all sufficiently large x . **17.** There are constants C_1, C_2, k_1 , and k_2 such that $|f(x)| \leq C_1|g(x)|$ for all $x > k_1$ and $|g(x)| \leq C_2|h(x)|$ for all $x > k_2$. Hence, for $x > \max(k_1, k_2)$ it follows that $|f(x)| \leq C_1|g(x)| \leq C_1C_2|h(x)|$. This shows that $f(x)$ is $O(h(x))$. **19.** 2^{n+1} is $O(2^n)$; 2^{2n} is not. **21.** $1000 \log n$; $\sqrt{n}$; $n \log n$; $n^2/1,000,000$; 2^n ; 3^n ; $2n!$ **23.** The algorithm that uses $n \log n$ operations **25. a)** $O(n^3)$ **b)** $O(n^5)$ **c)** $O(n^3 \cdot n!)$ **27. a)** $O(n^2 \log n)$ **b)** $O(n^2(\log n)^2)$ **c)** $O(n^{2^n})$ **29. a)** Neither $\Theta(x^2)$ nor $\Omega(x^2)$ **b)** $\Theta(x^2)$ and $\Omega(x^2)$ **c)** Neither $\Theta(x^2)$ nor $\Omega(x^2)$ **d)** $\Omega(x^2)$, but not $\Theta(x^2)$ **e)** $\Omega(x^2)$, but not $\Theta(x^2)$ **f)** $\Omega(x^2)$ and $\Theta(x^2)$ **31.** If $f(x)$ is $\Theta(g(x))$, then there exist constants C_1 and C_2 with $C_1|g(x)| \leq |f(x)| \leq C_2|g(x)|$. It follows that $|f(x)| \leq C_2|g(x)|$ and $|g(x)| \leq (1/C_1)|f(x)|$ for $x > k$. Thus, $f(x)$ is $O(g(x))$ and $g(x)$ is $O(f(x))$. Conversely, suppose that $f(x)$ is $O(g(x))$ and $g(x)$ is $O(f(x))$. Then there are constants C_1, C_2, k_1 , and k_2 such that $|f(x)| \leq C_1|g(x)|$ for $x > k_1$ and $|g(x)| \leq C_2|f(x)|$ for $x > k_2$. We can assume that $C_2 > 0$ (we can always make C_2 larger). Then we have $(1/C_2)|g(x)| \leq |f(x)| \leq C_1|g(x)|$ for $x > \max(k_1, k_2)$. Hence, $f(x)$ is $\Theta(g(x))$. **33.** If $f(x)$ is $\Theta(g(x))$, then $f(x)$ is both $O(g(x))$ and $\Omega(g(x))$. Hence, there are positive constants C_1, k_1, C_2 , and k_2 such that $|f(x)| \leq C_2|g(x)|$ for all $x > k_2$ and $|f(x)| \geq C_1|g(x)|$ for all $x > k_1$. It follows that $C_1|g(x)| \leq |f(x)| \leq C_2|g(x)|$ whenever $x > k$, where $k = \max(k_1, k_2)$. Conversely, if there are positive constants C_1, C_2 , and k such that $C_1|g(x)| \leq |f(x)| \leq C_2|g(x)|$ for $x > k$, then taking $k_1 = k_2 = k$ shows that $f(x)$ is both $O(g(x))$ and $\Theta(g(x))$.

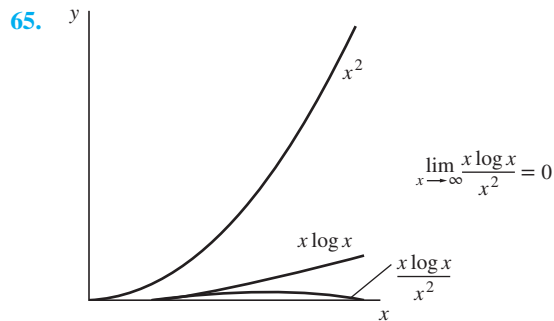


37. If $f(x)$ is $\Theta(1)$, then $|f(x)|$ is bounded between positive constants C_1 and C_2 . In other words, $f(x)$ cannot grow larger than a fixed bound or smaller than the negative of this bound and must not get closer to 0 than some fixed bound. **39.** Because $f(x)$ is $O(g(x))$, there are constants C and k such that $|f(x)| \leq C|g(x)|$ for $x > k$. Hence, $|f^n(x)| \leq C^n|g^n(x)|$ for $x > k$, so $f^n(x)$ is $O(g^n(x))$ by taking the constant to be C^n . **41.** Because $f(x)$ and $g(x)$ are increasing and unbounded, we can assume $f(x) \geq 1$ and $g(x) \geq 1$ for sufficiently large x .

There are constants C and k with $f(x) \leq Cg(x)$ for $x > k$. This implies that $\log f(x) \leq \log C + \log g(x) < 2 \log g(x)$ for sufficiently large x . Hence, $\log f(x)$ is $O(\log g(x))$. **43.** By definition there are positive constraints $C_1, C'_1, C_2, C'_2, k_1, k'_1, k_2$, and k'_2 such that $f_1(x) \geq C_1|g(x)|$ for all $x > k_1$, $f_1(x) \leq C'_1|g(x)|$ for all $x > k'_1$, $f_2(x) \geq C_2|g(x)|$ for all $x > k_2$, and $f_2(x) \leq C'_2|g(x)|$ for all $x > k'_2$. Adding the first and third inequalities shows that $f_1(x) + f_2(x) \geq (C_1 + C_2)|g(x)|$ for all $x > k$ where $k = \max(k_1, k_2)$. Adding the second and fourth inequalities shows that $f_1(x) + f_2(x) \leq (C'_1 + C'_2)|g(x)|$ for all $x > k'$ where $k' = \max(k'_1, k'_2)$. Hence, $f_1(x) + f_2(x)$ is $\Theta(g(x))$. This is no longer true if f_1 and f_2 can assume negative values. **45.** This is false. Let $f_1 = x^2 + 2x, f_2(x) = x^2 + x$, and $g(x) = x^2$. Then $f_1(x)$ and $f_2(x)$ are both $O(g(x))$, but $(f_1 - f_2)(x)$ is not. **47.** Take $f(n)$ to be the function with $f(n) = n$ if n is an odd positive integer and $f(n) = 1$ if n is an even positive integer and $g(n)$ to be the function with $g(n) = 1$ if n is an odd positive integer and $g(n) = n$ if n is an even positive integer. **49.** There are positive constants $C_1, C_2, C'_1, C'_2, k_1, k'_1, k_2$, and k'_2 such that $|f_1(x)| \geq C_1|g_1(x)|$ for all $x > k_1$, $|f_1(x)| \leq C'_1|g_1(x)|$ for all $x \geq k'_1$, $|f_2(x)| \geq C_2|g_2(x)|$ for all $x > k_2$, and $|f_2(x)| \leq C'_2|g_2(x)|$ for all $x > k'_2$. Because f_2 and g_2 are never zero, the last two inequalities can be rewritten as $|1/f_2(x)| \leq (1/C_2)|1/g_2(x)|$ for all $x > k_2$ and $|1/f_2(x)| \geq (1/C'_2)|1/g_2(x)|$ for all $x > k'_2$. Multiplying the first and rewritten fourth inequalities shows that $|f_1(x)/f_2(x)| \geq (C_1/C'_2)|g_1(x)/g_2(x)|$ for all $x > \max(k_1, k'_2)$, and multiplying the second and rewritten third inequalities gives $|f_1(x)/f_2(x)| \leq (C'_1/C_2)|g_1(x)/g_2(x)|$ for all $x > \max(k'_1, k_2)$. It follows that f_1/f_2 is big-Theta of g_1/g_2 . **51.** There exist positive constants $C_1, C_2, k_1, k_2, k'_1, k'_2$ such that $|f(x, y)| \leq C_1|g(x, y)|$ for all $x > k_1$ and $y > k_2$ and $|f(x, y)| \geq C_2|g(x, y)|$ for all $x > k'_1$ and $y > k'_2$. **53.** $(x^2 + xy + x \log y)^3 < (3x^2y^3) = 27x^6y^3$ for $x > 1$ and $y > 1$, because $x^2 < x^2y, xy < x^2y$, and $x \log y < x^2y$. Hence, $(x^2 + xy + x \log y)^3$ is $O(x^6y^3)$. **55.** For all positive real numbers x and y , $[xy] \leq xy$. Hence, $[xy]$ is $O(xy)$ from the definition, taking $C = 1$ and $k_1 = k_2 = 0$. **57.** Clearly $n^d < n^c$ for all $n \geq 2$; therefore, n^d is $O(n^c)$. The ratio $n^d/n^c = n^{d-c}$ is unbounded so there is no constant C such that $n^d \leq Cn^c$ for large n . **59.** If f and g are positive-valued functions such that $\lim_{n \rightarrow \infty} f(x)/g(x) = C < \infty$, then $f(x) < (C + 1)g(x)$ for large enough x , so $f(n)$ is $O(g(n))$. If that limit is ∞ , then $f(n)$ is not $O(g(n))$. Here repeated applications of L'Hôpital's rule shows that $\lim_{x \rightarrow \infty} x^d/b^x = 0$ and $\lim_{x \rightarrow \infty} b^x/x^d = \infty$. **61.** To show that c^n is $O(n!)$, assume WLOG that c is an integer greater than 1. Claim that if $n \geq c^2 + c$, then $n! \geq c^n$. Both $n!$ and c^n have n factors. Replacing each of the factors from $c^2 + 1$ to $c^2 + c$ in $n!$ by c^2 only makes the product smaller. But then each of these factors c^2 can be factored as $c \cdot c$ and one of those factors of c moved to pair with the factors 1 through c in $n!$. At this point every factor of $n!$ is greater than or equal to c , so the product is greater than or equal to c^n . To show that $n!$ is not $O(c^n)$, look at the ratio $n!/c^n$ and write this as $(c!/c^c) \cdot ((c+1)/c) \cdot ((c+2)/c) \cdot ((c+3)/c) \cdots ((2c)/c) \cdot ((2c+1)/c) \cdots (n/c)$. Treat the product of the

first $c + 1$ factors as a constant and note that every other factor is greater than 2. Thus, the product is greater than an arbitrarily large power of 2 as $n \rightarrow \infty$, which therefore approaches ∞ . Therefore, $n!$ cannot be bounded by a constant times c^n .

63. a) $\lim_{x \rightarrow \infty} x^2/x^3 = \lim_{x \rightarrow \infty} 1/x = 0$ **b)** $\lim_{x \rightarrow \infty} \frac{x \log x}{x^2} = \lim_{x \rightarrow \infty} \frac{\log x}{x} = \lim_{x \rightarrow \infty} \frac{1}{x \ln 2} = 0$ (using L'Hôpital's rule)
c) $\lim_{x \rightarrow \infty} \frac{x^2}{2^x} = \lim_{x \rightarrow \infty} \frac{2x}{2^x \ln 2} = \lim_{x \rightarrow \infty} \frac{2}{2^x (\ln 2)^2} = 0$ (using L'Hôpital's rule) **d)** $\lim_{x \rightarrow \infty} \frac{x^2 + x + 1}{x^2} = \lim_{x \rightarrow \infty} \left(1 + \frac{1}{x} + \frac{1}{x^2}\right) = 1 \neq 0$



67. No. Take $f(x) = 1/x^2$ and $g(x) = 1/x$. **69. a)** Because $\lim_{x \rightarrow \infty} f(x)/g(x) = 0$, $|f(x)|/|g(x)| < 1$ for sufficiently large x . Hence, $|f(x)| < |g(x)|$ for $x > k$ for some constant k . Therefore, $f(x)$ is $O(g(x))$. **b)** Let $f(x) = g(x) = x$. Then $f(x)$ is $O(g(x))$, but $f(x)$ is not $o(g(x))$ because $f(x)/g(x) = 1$. **71.** Because $f_2(x)$ is $o(g(x))$, from Exercise 69(a) it follows that $f_2(x)$ is $O(g(x))$. By Corollary 1, we have $f_1(x) + f_2(x)$ is $O(g(x))$. **73.** We can show that $(n - i)(i + 1) \geq n$ for $i = 0, 1, \dots, n - 1$. Hence, $(n!)^2 = (n \cdot 1)((n - 1) \cdot 2) \cdot ((n - 2) \cdot 3) \cdots (2 \cdot (n - 1)) \cdot (1 \cdot n) \geq n^n$. Therefore, $2 \log n! \geq n \log n$. **75.** Compute that $\log 5! \approx 6.9$ and $(5 \log 5)/4 \approx 2.9$, so the inequality holds for $n = 5$. Assume $n \geq 6$. Because $n!$ is the product of all the integers from n down to 1, we have $n! > n(n - 1)(n - 2) \cdots \lceil n/2 \rceil$ (because at least the term 2 is missing). Note that there are more than $n/2$ terms in this product, and each term is at least as big as $n/2$. Therefore, the product is greater than $(n/2)^{(n/2)}$. Taking the log of both sides of the inequality, we have $\log n! > \log \left(\frac{n}{2}\right)^{n/2} = \frac{n}{2} \log \frac{n}{2} = \frac{n}{2} (\log n - 1) > (n \log n)/4$, because $n > 4$ implies $\log n - 1 > (\log n)/2$. **77.** All are not asymptotic.

Section 3.3

- 1.** $O(1)$ **3.** $O(n^2)$ **5.** $2n - 1$ **7.** Linear **9.** $O(n)$
11. a) procedure *disjointpair*($S_1, S_2, \dots, S_n$:
subsets of $\{1, 2, \dots, n\}$)
 answer := false
 for $i := 1$ to n
 for $j := i + 1$ to n
 disjoint := true

for $k := 1$ to n
 if $k \in S_i$ and $k \in S_j$ then *disjoint* := false
 if *disjoint* then *answer* := true
return *answer*

- b)** $O(n^3)$ **13. a)** $\text{power} := 1, y := 1; i := 1, \text{power} := 2, y := 3; i := 2, \text{power} := 4, y := 15$ **b)** $2n$ multiplications and n additions **15. a)** $2^{10^9} \approx 10^{3 \times 10^8}$ **b)** 10^9 **c)** 3.96×10^7
d) 3.16×10^4 **e)** 29 **f)** 12 **17. a)** $2^{2^{60 \cdot 10^{12}}}$ **b)** $2^{60 \cdot 10^{12}}$
c) $\lfloor 2^{\sqrt{60 \cdot 10^6}} \rfloor \approx 2 \times 10^{2331768}$ **d)** 60,000,000 **e)** 7,745,966
f) 45 **g)** 6 **19. a)** 36 years **b)** 13 days **c)** 19 minutes
21. a) Less than 1 millisecond more **b)** 100 milliseconds more **c)** $2n + 1$ milliseconds more **d)** $3n^2 + 3n + 1$ milliseconds more **e)** Twice as much time **f)** 2^{2n+1} times as many milliseconds **g)** $n + 1$ times as many milliseconds **23.** The average number of comparisons is $(3n + 4)/2$. **25.** $O(\log n)$
27. $O(n)$ **29.** $O(n^2)$ **31.** $O(n)$ **33.** $O(n)$ **35.** $O(\log n)$ comparisons; $O(n^2)$ swaps **37.** $O(m(n - m))$ **39. a)** $O(n^2)$ **b)** $O(n \log n)$ **41.** $O(n^2 2^n)$ **43. a)** doubles **b)** increases by 1 **45.** Use Algorithm 1, where **A** and **B** are now $n \times n$ upper triangular matrices, by replacing m by n in line 1, and having q iterate only from i to j , rather than from 1 to k .
47. $n(n + 1)(n + 2)/6$ **49.** **A**((**BC**)**D**)

Supplementary Exercises

- 1. a)** procedure *last max*($a_1, \dots, a_n$; integers)
 max := a_1
 last := 1
 i := 2
 while $i \leq n$
 if $a_i \geq \text{max}$ then
 max := a_i
 last := i
 i := $i + 1$
 return *last*
b) $2n - 1 = O(n)$ comparisons
3. a) procedure *pair zeros*($b_1 b_2 \dots b_n$; bit string, $n \geq 2$)
 x := b_1
 y := b_2
 k := 2
 while $k < n$ and ($x \neq 0$ or $y \neq 0$)
 k := $k + 1$
 x := y
 y := b_k
 if $x = 0$ and $y = 0$ then print "YES"
 else print "NO"
b) $O(n)$
5. a) and **b)**
procedure *smallest and largest*($a_1, a_2, \dots, a_n$; integers)
 min := a_1
 max := a_1

S-24 Answers to Odd-Numbered Exercises

for $i := 2$ to n

if $a_i < \min$ then $\min := a_i$

if $a_i > \max$ then $\max := a_i$

{ $\min$ is the smallest integer among the input, and $\max$ is the largest}

c) $2n - 2$

7. Before any comparisons are done, there is a possibility that each element could be the maximum and a possibility that it could be the minimum. This means that there are $2n$ different possibilities, and $2n - 2$ of them have to be eliminated through comparisons of elements, because we need to find the unique maximum and the unique minimum. We classify comparisons of two elements as “virgin” or “nonvirgin,” depending on whether or not both elements being compared have been in any previous comparison. A virgin comparison eliminates the possibility that the larger one is the minimum and that the smaller one is the maximum; thus, each virgin comparison eliminates two possibilities, but it clearly cannot do more. A nonvirgin comparison must be between two elements that are still in the running to be the maximum or two elements that are still in the running to be the minimum, and at least one of these elements must *not* be in the running for the other category. For example, we might be comparing x and y , where all we know is that x has been eliminated as the minimum. If we find that $x > y$ in this case, then only one possibility has been ruled out—we now know that y is not the maximum. Thus, in the worst case, a nonvirgin comparison eliminates only one possibility. (The cases of other nonvirgin comparisons are similar.) Now there are at most $\lfloor n/2 \rfloor$ comparisons of elements that have not been compared before, each removing two possibilities; they remove $2\lfloor n/2 \rfloor$ possibilities altogether. Therefore, we need $2n - 2 - 2\lfloor n/2 \rfloor$ more comparisons that, as we have argued, can remove only one possibility each, in order to find the answers in the worst case, because $2n - 2$ possibilities have to be eliminated. This gives us a total of $2n - 2 - 2\lfloor n/2 \rfloor + \lfloor n/2 \rfloor$ comparisons in all. But $2n - 2 - 2\lfloor n/2 \rfloor + \lfloor n/2 \rfloor = 2n - 2 - \lfloor n/2 \rfloor = 2n - 2 + \lceil -n/2 \rceil = \lceil 2n - n/2 \rceil - 2 = \lceil 3n/2 \rceil - 2$, as desired. 9. The following algorithm has worst-case complexity $O(n^4)$.

procedure *equal sums*($a_1, a_2, \dots, a_n$)

for $i := 1$ to n

for $j := i + 1$ to n {since we want $i < j$ }

for $k := 1$ to n

for $l := k + 1$ to n {since we want $k < l$ }

if $a_i + a_j = a_k + a_l$ and $(i, j) \neq (k, l)$

then output these pairs

11. At end of first pass: 3, 1, 4, 5, 2, 6; at end of second pass: 1, 3, 2, 4, 5, 6; at end of third pass: 1, 2, 3, 4, 5, 6; fourth pass finds nothing to exchange and algorithm terminates 13. There are possibly as many as n passes through the list, and each pass uses $O(n)$ comparisons. Thus, there are $O(n^2)$ comparisons in all. 15. Because $\log n < n$, we have $(n \log n + n^2)^3 \leq (n^2 + n^2)^3 \leq (2n^2)^3 = 8n^6$ for all $n > 0$. This proves that $(n \log n + n^2)^3$ is $O(n^6)$, with witnesses $C = 8$ and $k = 0$. 17. $O(x^2 2^x)$ 19. Note that $\frac{n!}{2^n} = \frac{n}{2} \cdot \frac{n-1}{2} \cdots \frac{3}{2} \cdot \frac{2}{2} \cdot \frac{1}{2} >$

$\frac{n}{2} \cdot 1 \cdot 1 \cdots 1 \cdot \frac{1}{2} = \frac{n}{4}$. 21. All of these functions are of the same order. 23. 2^{107} 25. $(\log n)^2$, $2^{\sqrt{\log_2 n}}$, $n(\log n)^{1001}$, $n^{1.0001}$, 1.0001^n , n^n 27. For example, $f(n) = n^{2\lfloor n/2 \rfloor + 1}$ and $g(n) = n^{2\lceil n/2 \rceil}$

29. a) **procedure** *brute*($a_1, a_2, \dots, a_n$: integers)

for $i := 1$ to $n - 1$

for $j := i + 1$ to n

for $k := 1$ to n

if $a_i + a_j = a_k$ then return true

else return false

b) $O(n^3)$

31. For m_1 : w_1 and w_2 ; for m_2 : w_1 and w_3 ; for m_3 : w_2 and w_3 ; for w_1 : m_1 and m_2 ; for w_2 : m_1 and m_3 ; for w_3 : m_2 and m_3

33. A matching in which each woman is assigned her valid partner ranking highest on her preference list is female optimal; a matching in which each man is assigned his valid partner ranking lowest on his preference list is male pessimal.

35. a) Modify the preamble to Exercise 64 in Section 3.1 so that there are s men $m_1, m_2, \dots, m_s$ and t women $w_1, w_2, \dots, w_t$. A matching will contain $\min(s, t)$ marriages. The definition of “stable marriage” is the same, with the understanding that each person prefers any mate to being unmatched. b) Create $|s - t|$ fictitious people (men or women, whichever is in shorter supply) so that the number of men and the number of women become the same, and put these fictitious people at the bottom of everyone’s preference lists.

c) This follows immediately from Exercise 67 in Section 3.1.

37. 5; 15 39. The first situation in Exercise 37 41. a) For each subset S of $\{1, 2, \dots, n\}$, compute $\sum_{j \in S} w_j$. Keep track of the subset giving the largest such sum that is less than or equal to W , and return that subset as the output of the algorithm. b) The food pack and the portable stove

43. a) The makespan is always at least as large as the load on the processor assigned to do the lengthiest job, which must be at least $\max_{j=1,2,\dots,n} t_j$. Therefore, the minimum makespan satisfies this inequality. b) The total amount of time the processors need to spend working on the jobs (the total load) is $\sum_{j=1}^n t_j$. Therefore, the average load per processor is $\frac{1}{p} \sum_{j=1}^n t_j$. The maximum load cannot be any smaller than the average, so the minimum makespan is always at least this large.

45. Processor 1: jobs 1, 4; processor 2: job 2; processor 3: jobs 3, 5

CHAPTER 4

Section 4.1

1. a) Yes b) No c) Yes d) No 3. Suppose that $a \mid b$. Then there exists an integer k such that $ka = b$. Because $a(ck) = bc$ it follows that $a \mid bc$. 5. If $a \mid b$ and $b \mid a$, there are integers c and d such that $b = ac$ and $a = bd$. Hence, $a = acd$. Because $a \neq 0$ it follows that $cd = 1$. Thus, either $c = d = 1$ or $c = d = -1$. Hence, either $a = b$ or $a = -b$. 7. Because $ac \mid bc$ there is an integer k such